

Projet RFID

Suivi Chariots & Automatisation Oracle

Présentation IT — Phase de développement & Architecture finale

49

Chariots
total

98

Badges
RFID HF

4

Types de
chariots

3

Phases de
développement

Prototype actuel

Développement sur Raspberry Pi → réseau WiFi local
Même code utilisé en production sur serveur Linux GE

Technologies utilisées

Python

Flask · Paho-MQTT · MySQL-connector

MySQL

Base de données rfid_buc (8 tables)

Mosquitto

Broker MQTT — gestion messages RFID

Bootstrap 5

Interface responsive tablettes

JavaScript

Dashboard dynamique temps réel

paho-mqtt

Communication Pepper C1 → serveur

Architecture 2 couches

Couche 1 — Tracking physique

100% AUTOMATIQUE

Badge START detecte → statut EN_ATTENTE

Badge END detecte → statut RETOUR

Aucune action humaine requise

Couche 2 — Suivi jobs Oracle

VALIDATION HUMAINE UNIQUE :

Associer le job au chariot sur tablette WS

Badge START → Move Oracle automatique

Badge END → Aucune transaction Oracle

MQTT / MQTTS

Pepper C1 → Serveur

- ▶ Communication entre le lecteur RFID et le serveur
- ▶ Prototype → port 1883 (sans TLS)
- ▶ Production → port 8883 (MQTTS + TLS)
- ▶ Topic : rfid/buc/#
- ▶ QoS 1 — livraison garantie

HTTP / HTTPS

Tablettes → Serveur

- ▶ Accès dashboard web depuis les tablettes WS
- ▶ Prototype → port 5000 HTTP (Flask)
- ▶ Production → port 443 HTTPS (Nginx)
- ▶ Bootstrap 5 + JavaScript
- ▶ Refresh automatique toutes les 30 secondes

Telnet MWA MSCA

Automatisation Oracle

- ▶ Python simule la saisie clavier dans MSCA Oracle
- ▶ Connexion Telnet → Login → Navigation → Save
- ▶ Move Transaction WIP sans action humaine
- ▶ File attente MySQL → traitement asynchrone
- ▶ Approche validee : Hino GE Healthcare Japon

MySQL + sync_oracle

Données & Synchronisation

- ▶ MySQL : stockage missions, chariots, jobs, events
- ▶ sync_oracle.py : OFs Released → MySQL (periodique)
- ▶ oracle_move_queue : file attente transactions Oracle
- ▶ rfid_buc : 8 tables, schéma complet
- ▶ Connexion Oracle via cx_Oracle (GLTEST/GLPROD)

Architecture Finale

Serveur Linux GE | Sécurité | Cloud optionnel



Serveur Linux GE

- ▶ VM Ubuntu Server 22.04 LTS
- ▶ IP fixe — reseau usine GE Buc
- ▶ Mosquitto port 8883 (MQTTS+TLS)
- ▶ Flask + Nginx port 443 (HTTPS)
- ▶ MySQL base rfid_buc
- ▶ Meme code que Raspberry Pi
- ▶ Juste changer la configuration

Securite

- ▶ MQTTS port 8883 — TLS/SSL
- ▶ HTTPS port 443 — certificat SSL
- ▶ Username + Password Oracle
- ▶ VPN GE — acces externe securise
- ▶ Reseau local usine GE (LAN)
- ▶ Pas d'accès Internet requis
- ▶ Conforme securite GE Healthcare

Cloud GE (optionnel)

- ▶ Power BI — dashboard direction
- ▶ Alertes Microsoft Teams
- ▶ Sauvegarde MySQL automatique
- ▶ Accès hors usine (sans VPN)
- ▶ Monitoring + reporting KPIs
- ▶ Decision IT + Management
- ▶ Non requis pour le fonctionnement

Besoins IT — Ce que nous demandons

SERVEUR	VM Ubuntu Server 22.04 LTS IP fixe reseau usine GE Buc — meme configuration que Raspberry Pi	REQUIS
RESEAU	Ports 8883 et 443 ouverts MQTTS port 8883 (Pepper C1 → serveur) HTTPS port 443 (tablettes → serveur)	REQUIS
WIFI	Couverture WiFi industrielle SMK + ligne Pristina complète — Pepper C1 + tablettes WS + tablettes opérateurs	REQUIS
ORACLE	Accès Oracle GLTEST Username + Password MWA MSCA Organisation BUC — pour phase de tests	REQUIS
ORACLE	Accès Oracle GLPROD Même configuration — pour mise en production finale	REQUIS
SSL	Certificat SSL Pour HTTPS (Nginx) et MQTTS (Mosquitto) — auto-signé ou Let's Encrypt	REQUIS
VPN	VPN GE Healthcare Acces dashboard depuis hors usine — déjà existant dans GE	EXISTANT
CLOUD	Connecteur Power BI → MySQL Dashboard KPIs pour direction — si dashboard externe souhaité	OPTIONNEL